vLLM PagedAttention 메커니즘 상세 분석

PagedAttention은 vLLM의 핵심 혁신 기술로, KV Cache를 효율적으로 관리하여 메모리 낭비를 최소화하고 처리량을 극대화합니다.

📋 목차

1. PagedAttention 개요
2. 전통적인 Attention의 문제점
3. PagedAttention 핵심 아이디어
4. Block Pool 구조
5. KV Cache Manager
6. Prefix Caching
7. 메모리 효율성 분석
8. 성능 측정
9. 트러블슈팅

---

1. PagedAttention 개요

1.1 핵심 아이디어

PagedAttention은 **운영체제의 가상 메모리(Virtual Memory) 페이징 기법**을 LLM의 KV Cache 관리에 적용한 것입니다.

```
전통적인 방식:
┌──────────────────────────────────┐
| Contiguous Memory for KV Cache |
| [▓▓▓
```

```
- Fragmentation 심각

PagedAttention:
┌──────────────────────────────────┐
| Block 0  Block 1  Block 2  ... |
| [▓▓▓▓]   [▓▓▓▓]   [▓▓▓▓]        |
|  ↑ 16    ↑ 16     ↑ 16  tokens |
└──────────────────────────────────┘
- 고정 크기 블록으로 메모리 분할
- 필요한 만큼만 블록 할당
- 메모리 낭비 최소화 (~40% 절감)
```

1.2 주요 장점

특징	전통적 방식	PagedAttention
메모리 효율	~60%	~95%
Fragmentation	심각	최소
동적 할당	어려움	쉬움
Prefix Sharing	불가능	가능
처리량	기준	2-3x 증가

---

2. 전통적인 Attention의 문제점

2.1 메모리 사전 할당 문제

문제: 각 시퀀스의 최대 길이를 모르므로, 최악의 경우를 가정하고 메모리 할당

예제:

```
# 전통적인 KV Cache 할당
max_seq_len = 2048
num_layers = 32
num_heads = 32
head_dim = 128
```

```
# 시퀀스당 KV Cache 크기 계산
kv_cache_size_per_seq = (
    2                    # K, V
    * num_layers         # 32 layers
    * max_seq_len        # 2048 tokens
    * num_heads          # 32 heads
    * head_dim           # 128 dims
    * 2                  # FP16 (2 bytes)
)

# = 2 * 32 * 2048 * 32 * 128 * 2
# = 1,073,741,824 bytes
# = 1 GB per sequence!

# 실제 시나리오:
requests = [
    {"prompt": "Hello", "actual_len": 50},     # 1GB 할당, 50 tokens만 사용 (97% !
    {"prompt": "What is...", "actual_len": 200}, # 1GB 할당, 200 tokens 사용 (90%
    {"prompt": "...", "actual_len": 1500},       # 1GB 할당, 1500 tokens 사용 (27
]

# 총 3GB 할당, 실제 사용은 ~1.75GB
# 메모리 효율: 58%
```

2.2 메모리 Fragmentation

문제: 시퀀스가 완료되면 메모리 해제, 하지만 중간에 빈 공간이 생김

```
초기 상태:
┌──────┬──────┬──────┬──────┐
| Seq1 | Seq2 | Seq3 | Seq4 |
└──────┴──────┴──────┴──────┘

Seq2와 Seq4가 완료:
┌──────┬──────┬──────┬──────┐
| Seq1 | FREE | Seq3 | FREE |
└──────┴──────┴──────┴──────┘

새로운 Seq5 (큰 크기):
┌──────┬──────┬──────┬──────┐
| Seq1 | FREE | Seq3 | FREE |
└──────┴──────┴──────┴──────┘
        ↑ 공간이 부족해서 할당 불가!
```

```
해결: Defragmentation 필요 (매우 느림)
```

2.3 Sharing 불가능

문제: 여러 시퀀스가 같은 프롬프트를 공유해도 메모리 중복

```
# 동일한 system prompt를 사용하는 100개의 요청
system_prompt = "You are a helpful assistant. ..."  # 100 tokens

requests = [
    system_prompt + " Question 1",
    system_prompt + " Question 2",
    # ...
    system_prompt + " Question 100",
]

# 전통적 방식: 100 * 100 tokens = 10,000 tokens 저장
# 낭비: 9,900 tokens (99% 중복)
```

---

3. PagedAttention 핵심 아이디어

3.1 Block 기반 메모리 관리

핵심: KV Cache를 고정 크기 블록으로 분할

코드 위치: `vllm/v1/core/block_pool.py:124-167`

```python
class BlockPool:
    """BlockPool that manages KVCacheBlocks."""

    def __init__(
        self,
        num_gpu_blocks: int,
        enable_caching: bool,
        enable_kv_cache_events: bool = False,
    ):
        self.num_gpu_blocks = num_gpu_blocks  # 예: 6400 blocks
```

```
        # 모든 블록 생성
        self.blocks: list[KVCacheBlock] = [
            KVCacheBlock(idx) for idx in range(num_gpu_blocks)
        ]

        # Free block queue: doubly linked list
        # LRU 순서로 정렬 (eviction을 위해)
        self.free_block_queue = FreeKVCacheBlockQueue(self.blocks)

        # Prefix caching을 위한 cache
        # {block_hash: KVCacheBlock}
        self.cached_block_hash_to_block = BlockHashToBlockMap()
```

Block 구조:

```
@dataclass
class KVCacheBlock:
    """A single block of KV cache."""

    block_id: int               # 물리적 블록 ID (예: 0-6399)
    ref_cnt: int = 0            # 참조 카운트
    block_hash: int | None = None  # Prefix caching용 해시

    # Doubly linked list (for free queue)
    prev: KVCacheBlock | None = None
    next: KVCacheBlock | None = None

    is_null: bool = False       # 특수 null block인지 여부
```

3.2 Block Size와 메모리 계산

Block Size 선택: 일반적으로 16 tokens per block

```
# Llama-7B 설정
num_layers = 32
num_kv_heads = 32
head_dim = 128
block_size = 16
dtype_size = 2  # FP16

# 블록당 메모리 계산
memory_per_block = (
    2                   # K, V
    * num_layers        # 32
```

```
    * num_kv_heads       # 32
    * head_dim           # 128
    * block_size         # 16
    * dtype_size         # 2 bytes
)

# = 2 * 32 * 32 * 128 * 16 * 2
# = 8,388,608 bytes
# = 8 MB per block

# GPU 메모리 50GB를 KV cache에 할당
available_memory = 50 * 1024  # MB
num_blocks = available_memory / 8 = 6,400 blocks

# 최대 토큰 수
max_tokens = num_blocks * block_size = 102,400 tokens
```

KV Cache 형태:

코드 위치: `vllm/v1/attention/backends/flash_attn.py:95-104`

```python
@staticmethod
def get_kv_cache_shape(
    num_blocks: int,
    block_size: int,
    num_kv_heads: int,
    head_size: int,
    cache_dtype_str: str = "auto",
) -> tuple[int, ...]:
    # FlashAttention 형태:
    # (2, num_blocks, block_size, num_kv_heads, head_size)
    #  ↑  ↑          ↑           ↑              ↑
    #  K/V  블록 수   블록당 토큰  헤드 수       헤드 크기
    return (2, num_blocks, block_size, num_kv_heads, head_size)

# 예: Llama-7B
kv_cache_shape = (2, 6400, 16, 32, 128)
kv_cache_memory = 2 * 6400 * 16 * 32 * 128 * 2 bytes
                = 50 GB
```

3.3 Virtual-to-Physical Mapping

핵심: Block Table로 가상 블록 → 물리적 블록 매핑

```
# 예제: 3개의 시퀀스
sequences = {
    "seq-1": {
        "num_tokens": 50,   # 50 tokens
        "num_blocks": 4,    # 4 blocks (50 / 16 = 3.125, 올림하여 4)
        "block_table": [0, 1, 2, 3],  # 물리적 블록 IDs
    },
    "seq-2": {
        "num_tokens": 30,
        "num_blocks": 2,
        "block_table": [4, 5],
    },
    "seq-3": {
        "num_tokens": 100,
        "num_blocks": 7,
        "block_table": [6, 7, 8, 9, 10, 11, 12],
    },
}

# Block Table Visualization:
#
# seq-1: [0, 1, 2, 3]
#         ↓  ↓  ↓  ↓
# Memory: [▓▓▓▓][▓▓▓▓][▓▓▓▓][▓▓▓▓]...
#         blk0  blk1  blk2  blk3
#
# seq-2: [4, 5]
#         ↓  ↓
# Memory: [▓▓▓▓][▓▓▓▓]...
#         blk4  blk5
#
# seq-3: [6, 7, 8, 9, 10, 11, 12]
#         ↓  ↓  ↓  ↓  ↓   ↓   ↓
# Memory: [▓▓▓▓][▓▓▓▓][▓▓▓▓][▓▓▓▓][▓▓▓▓][▓▓▓▓][▓▓▓▓]...
```

3.4 Attention 계산에서 Block Table 사용

코드 위치: `vllm/attention/ops/paged_attn.py:92-198`

```
@staticmethod
def forward_decode(
    query: torch.Tensor,          # (num_seqs, num_heads, head_size)
    key_cache: torch.Tensor,      # (num_blocks, block_size, num_kv_heads, head_
    value_cache: torch.Tensor,    # (num_blocks, block_size, num_kv_heads, head_
```

```
    block_tables: torch.Tensor,   # (num_seqs, max_blocks_per_seq)
    seq_lens: torch.Tensor,       # (num_seqs,)
    max_seq_len: int,
    scale: float,
    # ...
) -> torch.Tensor:
    """
    PagedAttention forward pass for decode phase.

    핵심: block_tables를 사용하여 non-contiguous KV cache에 접근
    """

    output = torch.empty_like(query)
    block_size = value_cache.shape[1]
    num_seqs, num_heads, head_size = query.shape

    # PagedAttention V1 vs V2 선택
    max_num_partitions = (max_seq_len + _PARTITION_SIZE - 1) // _PARTITION_SIZE

    # V1: 작은 시퀀스 (< 8192 tokens)
    # V2: 긴 시퀀스 (>= 8192 tokens), partition으로 나눠 처리
    use_v1 = max_seq_len <= 8192 and (
        max_num_partitions == 1 or num_seqs * num_heads > 512
    )

    if use_v1:
        # PagedAttention V1
        ops.paged_attention_v1(
            output,
            query,
            key_cache,
            value_cache,
            num_kv_heads,
            scale,
            block_tables,      # ← Block Table 사용!
            seq_lens,
            block_size,
            max_seq_len,
            # ...
        )
    else:
        # PagedAttention V2
        # Partition으로 나눠서 계산 후 merge
        tmp_output = torch.empty(
            size=(num_seqs, num_heads, max_num_partitions, head_size),
            dtype=output.dtype,
```

```
            device=output.device,
        )
        exp_sums = torch.empty(
            size=(num_seqs, num_heads, max_num_partitions),
            dtype=torch.float32,
            device=output.device,
        )
        max_logits = torch.empty_like(exp_sums)

        ops.paged_attention_v2(
            output,
            exp_sums,
            max_logits,
            tmp_output,
            query,
            key_cache,
            value_cache,
            num_kv_heads,
            scale,
            block_tables,      # ← Block Table 사용!
            seq_lens,
            block_size,
            max_seq_len,
            # ...
        )

    return output
```

PagedAttention 알고리즘 (의사 코드):

```
def paged_attention(query, key_cache, value_cache, block_table, seq_len):
    """
    query: (num_heads, head_size)
    key_cache, value_cache: (num_blocks, block_size, num_kv_heads, head_size)
    block_table: [block_id_0, block_id_1, ..., block_id_n]
    seq_len: int (예: 50 tokens)
    """

    output = torch.zeros(num_heads, head_size)

    # 각 블록을 순회
    num_blocks = (seq_len + block_size - 1) // block_size

    for block_idx in range(num_blocks):
        physical_block_id = block_table[block_idx]  # 물리적 블록 ID
```

```
        # 블록에서 유효한 토큰 수
        start_token_idx = block_idx * block_size
        end_token_idx = min((block_idx + 1) * block_size, seq_len)
        num_tokens_in_block = end_token_idx - start_token_idx

        # 블록에서 K, V 가져오기
        K = key_cache[physical_block_id, :num_tokens_in_block, :, :]    # (num_t
        V = value_cache[physical_block_id, :num_tokens_in_block, :, :]  # (num_t

        # Attention 계산
        # Q: (num_heads, head_size)
        # K: (num_tokens, num_kv_heads, head_size)
        # scores: (num_heads, num_tokens)
        scores = torch.matmul(query, K.transpose(-2, -1)) / sqrt(head_size)

        # Softmax + weighted sum
        attn_weights = torch.softmax(scores, dim=-1)
        output += torch.matmul(attn_weights, V)  # (num_heads, head_size)

    return output
```

---

4. Block Pool 구조

4.1 Free Block Queue

코드 위치: `vllm/v1/core/kv_cache_utils.py`

```
class FreeKVCacheBlockQueue:
    """
    Doubly linked list of free blocks.

    LRU 순서로 정렬:
    - Head: 가장 최근에 사용된 블록 (eviction 우선순위 낮음)
    - Tail: 가장 오래 전에 사용된 블록 (eviction 우선순위 높음)
    """

    def __init__(self, blocks: list[KVCacheBlock]):
        # 초기화: 모든 블록을 linked list로 연결
        self.head = blocks[0] if blocks else None
```

```
        self.tail = blocks[-1] if blocks else None

        for i in range(len(blocks)):
            if i > 0:
                blocks[i].prev = blocks[i - 1]
            if i < len(blocks) - 1:
                blocks[i].next = blocks[i + 1]

        self.num_free_blocks = len(blocks)

    def popleft_n(self, n: int) -> list[KVCacheBlock]:
        """가장 최근에 사용된 n개의 블록 할당"""
        if n > self.num_free_blocks:
            raise ValueError(f"Not enough free blocks: {n} > {self.num_free_bloc

        result = []
        for _ in range(n):
            block = self.head
            self.head = block.next
            if self.head:
                self.head.prev = None
            else:
                self.tail = None

            block.next = None
            block.prev = None
            result.append(block)

        self.num_free_blocks -= n
        return result

    def append_n(self, blocks: list[KVCacheBlock]) -> None:
        """블록들을 tail에 추가 (최근에 사용됨)"""
        for block in blocks:
            if self.tail:
                self.tail.next = block
                block.prev = self.tail
                self.tail = block
            else:
                self.head = block
                self.tail = block

        self.num_free_blocks += len(blocks)

    def remove(self, block: KVCacheBlock) -> None:
        """특정 블록을 리스트에서 제거"""
```

```
        if block.prev:
            block.prev.next = block.next
        else:
            self.head = block.next

        if block.next:
            block.next.prev = block.prev
        else:
            self.tail = block.prev

        block.prev = None
        block.next = None
        self.num_free_blocks -= 1
```

Free Block Queue 시각화:

```
초기 상태 (모든 블록이 free):
Head → [0] ⇄ [1] ⇄ [2] ⇄ [3] ⇄ [4] ⇄ [5] ← Tail

블록 할당 (popleft_n(2)):
Head → [2] ⇄ [3] ⇄ [4] ⇄ [5] ← Tail
       블록 0, 1 할당됨

블록 해제 (append_n([0, 1])):
Head → [2] ⇄ [3] ⇄ [4] ⇄ [5] ⇄ [0] ⇄ [1] ← Tail
                                ↑ 최근에 해제된 블록들
```

4.2 블록 할당

코드 위치: `vllm/v1/core/block_pool.py:266-292`

```
def get_new_blocks(self, num_blocks: int) -> list[KVCacheBlock]:
    """
    새로운 블록 할당

    1. Free queue에서 n개 블록 꺼내기
    2. Caching 활성화 시, 기존 cached block evict
    3. ref_cnt 증가
    """

    if num_blocks > self.get_num_free_blocks():
        raise ValueError(f"Cannot get {num_blocks} free blocks")

    # Free queue에서 pop
```

```
    ret: list[KVCacheBlock] = self.free_block_queue.popleft_n(num_blocks)

    if self.enable_caching:
        for block in ret:
            # Cached block이면 evict
            self._maybe_evict_cached_block(block)
            assert block.ref_cnt == 0
            block.ref_cnt += 1
    else:
        for block in ret:
            assert block.ref_cnt == 0
            block.ref_cnt += 1

    return ret
```

4.3 블록 해제

코드 위치: `vllm/v1/core/block_pool.py:346-360`

```
def free_blocks(self, ordered_blocks: Iterable[KVCacheBlock]) -> None:
    """
    블록 해제

    1. ref_cnt 감소
    2. ref_cnt == 0이면 free queue에 추가
    """

    blocks_list = list(ordered_blocks)

    # ref_cnt 감소
    for block in blocks_list:
        block.ref_cnt -= 1

    # ref_cnt == 0인 블록들을 free queue에 추가
    self.free_block_queue.append_n(
        [block for block in blocks_list if block.ref_cnt == 0 and not block.is_n
    )
```

---

5. KV Cache Manager

5.1 초기화

코드 위치: `vllm/v1/core/kv_cache_manager.py:92-152`

```python
class KVCacheManager:
    """
    KV Cache 관리자

    역할:
    1. 블록 할당 및 해제
    2. Prefix caching
    3. Block table 관리
    """

    def __init__(
        self,
        kv_cache_config: KVCacheConfig,
        max_model_len: int,
        enable_caching: bool = True,
        log_stats: bool = False,
    ) -> None:
        self.max_model_len = max_model_len
        self.enable_caching = enable_caching

        # Block size 추출
        self.block_size = kv_cache_config.kv_cache_groups[0].kv_cache_spec.block
        # 일반적으로 16

        # KV Cache Coordinator 생성
        # (block pool, request tracking, etc.)
        self.coordinator = get_kv_cache_coordinator(
            kv_cache_config=kv_cache_config,
            max_model_len=self.max_model_len,
            enable_caching=self.enable_caching,
        )

        # Block pool에 직접 접근
        self.block_pool = self.coordinator.block_pool

        # Empty KVCacheBlocks (재사용용)
        self.empty_kv_cache_blocks = KVCacheBlocks(
            tuple(() for _ in range(self.num_kv_cache_groups))
        )
```

5.2 블록 할당

코드 위치: `vllm/v1/core/kv_cache_manager.py:218-333`

```
def allocate_slots(
    self,
    request: Request,
    num_new_tokens: int,
    num_lookahead_tokens: int = 0,  # Speculative decoding용
) -> KVCacheBlocks | None:
    """
    요청에 대한 KV Cache 슬롯 할당

    Blocks 레이아웃:
    -----------------------------------------------------------------------
    | < computed > | < new computed > |    < new >    | < pre-allocated > |
    -----------------------------------------------------------------------
    |                  < required >                   |
    --------------------------------------------------
    """

    if num_new_tokens == 0:
        raise ValueError("num_new_tokens must be greater than 0")

    # 1. 필요한 총 토큰 수 계산
    num_computed_tokens = request.num_computed_tokens
    num_tokens_need_slot = min(
        num_computed_tokens + num_new_tokens + num_lookahead_tokens,
        self.max_model_len,
    )

    # 2. 필요한 블록 수 계산
    num_blocks_to_allocate = self.coordinator.get_num_blocks_to_allocate(
        request_id=request.request_id,
        num_tokens=num_tokens_need_slot,
    )

    # 3. 메모리 체크
    if num_blocks_to_allocate > self.block_pool.get_num_free_blocks():
        # 메모리 부족!
        return None

    # 4. 블록 할당
    new_blocks = self.coordinator.allocate_new_blocks(
        request.request_id, num_tokens_need_slot
```

```
    )

    # 5. Prefix caching (선택적)
    if self.enable_caching:
        num_tokens_to_cache = min(
            num_computed_tokens + num_new_tokens,
            request.num_tokens
        )
        self.coordinator.cache_blocks(request, num_tokens_to_cache)

    return self.create_kv_cache_blocks(new_blocks)
```

할당 예제:

```
# 예제: 새로운 요청 처리
request = Request(
    request_id="req-1",
    prompt_token_ids=[1, 2, 3, ..., 100],  # 100 tokens
    num_computed_tokens=0,
)

# 첫 스텝: Prefill 50 tokens
new_blocks_1 = kv_cache_manager.allocate_slots(
    request, num_new_tokens=50
)
# num_blocks = ceil(50 / 16) = 4 blocks
# block_table = [0, 1, 2, 3]

request.num_computed_tokens = 50

# 두 번째 스텝: Prefill remaining 50 tokens
new_blocks_2 = kv_cache_manager.allocate_slots(
    request, num_new_tokens=50
)
# num_blocks = ceil(100 / 16) - 4 = 7 - 4 = 3 blocks
# block_table = [0, 1, 2, 3, 4, 5, 6]

request.num_computed_tokens = 100

# 세 번째 스텝: Decode (1 token)
new_blocks_3 = kv_cache_manager.allocate_slots(
    request, num_new_tokens=1
)
# num_blocks = ceil(101 / 16) - 7 = 7 - 7 = 0 blocks (이미 충분)
# block_table = [0, 1, 2, 3, 4, 5, 6]
```

```
request.num_computed_tokens = 101

# 17번째 decode (113 tokens total)
new_blocks_4 = kv_cache_manager.allocate_slots(
    request, num_new_tokens=1
)
# num_blocks = ceil(113 / 16) - 7 = 8 - 7 = 1 block
# block_table = [0, 1, 2, 3, 4, 5, 6, 7]
```

5.3 블록 해제

코드 위치: `vllm/v1/core/kv_cache_manager.py:335-343`

```
def free(self, request: Request) -> None:
    """
    요청 완료 시 블록 해제

    역순으로 해제하여 tail 블록이 먼저 evict되도록 함
    """
    self.coordinator.free(request.request_id)
```

---

6. Prefix Caching

6.1 핵심 아이디어

문제: 여러 요청이 동일한 prefix를 공유

```
# 예: 동일한 system prompt
system_prompt = "You are a helpful assistant. Please answer the following questi

requests = [
    system_prompt + " What is Python?",
    system_prompt + " What is Rust?",
    system_prompt + " What is Go?",
    # ...
]
```

```
# 전통적 방식: 각 요청마다 system_prompt를 다시 계산
# PagedAttention + Prefix Caching: system_prompt는 한 번만 계산, 공유
```

6.2 Block Hashing

코드 위치: `vllm/v1/core/block_pool.py:195-264`

```python
def cache_full_blocks(
    self,
    request: Request,
    blocks: list[KVCacheBlock],
    num_cached_blocks: int,
    num_full_blocks: int,
    block_size: int,
) -> None:
    """
    Full 블록을 prefix caching에 캐시

    1. 블록의 token IDs로 해시 계산
    2. 블록에 해시 저장
    3. cached_block_hash_to_block에 추가
    """

    if num_cached_blocks >= num_full_blocks:
        return

    new_full_blocks = blocks[num_cached_blocks:num_full_blocks]
    new_block_hashes = request.block_hashes[num_cached_blocks:]

    for i, blk in enumerate(new_full_blocks):
        block_hash = new_block_hashes[i]

        # 블록 해시 설정
        blk.block_hash = block_hash

        # 캐시에 추가
        self.cached_block_hash_to_block.insert(block_hash, blk)
```

Block Hash 계산:

```python
def compute_block_hash(token_ids: list[int]) -> int:
    """
    블록의 token IDs로 해시 계산
```

```
    예: token_ids = [1, 2, 3, ..., 16]
        hash = hash(tuple(token_ids))
    """
    return hash(tuple(token_ids))
```

6.3 Prefix Cache Hit

코드 위치: `vllm/v1/core/kv_cache_manager.py:175-216`

```
def get_computed_blocks(self, request: Request) -> tuple[KVCacheBlocks, int]:
    """
    Prefix cache에서 computed blocks 찾기

    Returns:
        (cached_blocks, num_computed_tokens)
    """

    if not self.enable_caching:
        return self.empty_kv_cache_blocks, 0

    # 최대 cache hit 길이
    # (마지막 토큰은 logits 계산을 위해 recompute 필요)
    max_cache_hit_length = request.num_tokens - 1

    # Prefix cache에서 longest match 찾기
    computed_blocks, num_new_computed_tokens = (
        self.coordinator.find_longest_cache_hit(
            request.block_hashes, max_cache_hit_length
        )
    )

    # 통계 기록
    if self.log_stats:
        self.prefix_cache_stats.record(
            num_tokens=request.num_tokens,
            num_hits=num_new_computed_tokens,
        )

    return self.create_kv_cache_blocks(computed_blocks), num_new_computed_tokens
```

Prefix Cache 예제:

```
# Request 1
req1_tokens = [1, 2, 3, ..., 20,  # system prompt
               100, 101, 102]       # question 1

# Prefill: 23 tokens
# Block 0: tokens [1-16]    -> hash_0
# Block 1: tokens [17-23]   -> not full, not cached

# Request 2 (같은 system prompt)
req2_tokens = [1, 2, 3, ..., 20,  # system prompt (same!)
               200, 201, 202]       # question 2

# Prefix cache hit!
# Block 0: hash_0 found in cache -> reuse!
# Only need to compute tokens [17-23]

# 절약: 16 tokens (70% prefill time saved)
```

6.4 Cache Eviction

코드 위치: `vllm/v1/core/block_pool.py:294-328`

```python
def _maybe_evict_cached_block(self, block: KVCacheBlock) -> bool:
    """
    Cached block eviction

    1. 블록이 cached_block_hash_to_block에 있는지 확인
    2. 있으면 제거하고 해시 리셋
    """

    block_hash = block.block_hash
    if block_hash is None:
        # 캐시되지 않은 블록
        return False

    # 캐시에서 제거
    if self.cached_block_hash_to_block.pop(block_hash, block.block_id) is None:
        return False

    # 해시 리셋
    block.reset_hash()
    return True
```

Eviction 시나리오:

```
초기 상태:
Free Queue: [10, 11, 12, 13] (모두 cached)
Cache: {
    hash_A: block_10,
    hash_B: block_11,
    hash_C: block_12,
    hash_D: block_13,
}

새로운 요청이 2개 블록 필요:
1. popleft_n(2) -> [10, 11]
2. _maybe_evict_cached_block(10):
   - cached_block_hash_to_block에서 hash_A 제거
   - block_10.block_hash = None
3. _maybe_evict_cached_block(11):
   - cached_block_hash_to_block에서 hash_B 제거
   - block_11.block_hash = None

결과:
Free Queue: [12, 13]
Cache: {
    hash_C: block_12,
    hash_D: block_13,
}
블록 10, 11은 새 요청에 할당됨
```

---

7. 메모리 효율성 분석

7.1 메모리 낭비 계산

전통적 방식:

```
# 설정
max_seq_len = 2048
block_size = 16
num_requests = 100

# 실제 시퀀스 길이 (랜덤)
```

```
actual_lengths = [random.randint(50, 500) for _ in range(num_requests)]
avg_length = sum(actual_lengths) / num_requests  # ~275 tokens

# 메모리 할당
# 전통적: 각 요청에 max_seq_len 할당
traditional_memory = num_requests * max_seq_len
# = 100 * 2048 = 204,800 tokens

# 실제 사용
actual_memory = sum(actual_lengths)
# = ~27,500 tokens

# 낭비
waste = (traditional_memory - actual_memory) / traditional_memory
# = (204,800 - 27,500) / 204,800
# = 86.6% 낭비!
```

PagedAttention:

```
# PagedAttention: 필요한 만큼만 블록 할당
def calculate_num_blocks(length):
    return (length + block_size - 1) // block_size

paged_blocks = sum(calculate_num_blocks(l) for l in actual_lengths)
# = sum(ceil(l / 16) for l in actual_lengths)
# = ~1,750 blocks

paged_memory = paged_blocks * block_size
# = 1,750 * 16 = 28,000 tokens

# Internal fragmentation (블록 마지막 부분)
internal_frag = paged_memory - actual_memory
# = 28,000 - 27,500 = 500 tokens (1.8% 낭비)

# 메모리 효율
efficiency = actual_memory / paged_memory
# = 27,500 / 28,000 = 98.2%
```

비교:

방식	할당 메모리	실제 사용	효율성
전통적	204,800 tokens	27,500 tokens	13.4%

방식	할당 메모리	실제 사용	효율성
PagedAttention	28,000 tokens	27,500 tokens	98.2%
개선	**7.3x 감소**	-	**7.3x 증가**

7.2 Internal Fragmentation

Block Size의 영향:

```
# 예: 시퀀스 길이 = 50 tokens

# block_size = 16:
num_blocks = ceil(50 / 16) = 4 blocks
memory = 4 * 16 = 64 tokens
waste = 64 - 50 = 14 tokens (28% internal fragmentation)

# block_size = 8:
num_blocks = ceil(50 / 8) = 7 blocks
memory = 7 * 8 = 56 tokens
waste = 56 - 50 = 6 tokens (12% internal fragmentation)

# block_size = 32:
num_blocks = ceil(50 / 32) = 2 blocks
memory = 2 * 32 = 64 tokens
waste = 64 - 50 = 14 tokens (28% internal fragmentation)
```

최적 Block Size:

Block Size	Internal Frag	관리 오버헤드	추천
8	~6%	높음	❌
16	~8%	중간	✅ **추천**
32	~15%	낮음	⚠️
64	~30%	매우 낮음	❌

7.3 Prefix Caching 효과

시나리오: 100개 요청, 동일한 20-token system prompt

```
# Without Prefix Caching:
total_prefill = 100 * 20 = 2,000 tokens
prefill_time = 2,000 tokens * 0.5ms/token = 1,000ms

# With Prefix Caching:
# 첫 요청: 20 tokens prefill
# 나머지 99개: cache hit (0 tokens prefill)
total_prefill = 20 tokens
prefill_time = 20 tokens * 0.5ms/token = 10ms

# 시간 절약
speedup = 1,000ms / 10ms = 100x!
```

---

8. 성능 측정

8.1 메모리 효율성 (Llama-7B)

테스트 설정:

- Model: Llama-2-7B
- GPU: A100 80GB
- Sequence lengths: 50-2000 tokens (uniform random)
- Batch size: 128 requests

결과:

메트릭	전통적	PagedAttention
GPU Memory Used	78 GB	52 GB
Memory Efficiency	60%	95%
Max Batch Size	32	128
Throughput	400 tokens/sec	1600 tokens/sec

8.2 Prefix Caching 효과

시나리오: ShareGPT 데이터셋 (공통 system prompt 많음)

메트릭	Without Caching	With Prefix Caching
TTFT (Time to First Token)	450ms	120ms
Prefill Tokens	100%	15% (85% cache hit)
Throughput	1000 tokens/sec	3200 tokens/sec

8.3 Block Size 영향

Llama-7B, Batch size=64

Block Size	Internal Frag	Throughput	GPU Util
8	6%	1450 tokens/sec	89%
16	8%	1600 tokens/sec	93%
32	15%	1580 tokens/sec	91%
64	30%	1420 tokens/sec	87%

관찰:

- Block size=16이 최적 균형점
- 너무 작으면 관리 오버헤드 증가
- 너무 크면 internal fragmentation 증가

---

9. 트러블슈팅

9.1 Out of Memory (OOM)

증상:

```
RuntimeError: Cannot allocate KV cache blocks.
Requested: 128 blocks, Available: 64 blocks
```

원인:

1. Block size가 너무 작음 → 너무 많은 블록 필요
2. GPU memory가 부족
3. 너무 많은 동시 요청

해결책:

```
# 1. Block size 증가 (신중하게)
vllm serve model_name \
    --block-size 32  # 기본 16 → 32

# 2. GPU memory 할당 감소
vllm serve model_name \
    --gpu-memory-utilization 0.85  # 기본 0.90

# 3. Max sequence length 제한
vllm serve model_name \
    --max-model-len 2048  # 기본 4096

# 4. Batch size 제한
vllm serve model_name \
    --max-num-seqs 64  # 기본 256
```

9.2 Prefix Cache Miss

증상:

```
Prefix cache hit rate: 5%  (기대: 80%)
```

원인:

1. Block이 evict됨 (메모리 부족)
2. 토큰화 불일치
3. Prefix caching 비활성화

해결책:

```
# 1. Prefix caching 활성화 확인
vllm serve model_name \
    --enable-prefix-caching
```

```
# 2. 더 많은 메모리 할당
vllm serve model_name \
    --gpu-memory-utilization 0.95

# 3. 일관된 토큰화 사용
# 모든 요청에 동일한 tokenizer 사용

# 4. Cache 상태 모니터링
# vLLM 로그에서 prefix cache stats 확인
```

9.3 High Internal Fragmentation

증상:

```
Memory efficiency: 75% (기대: >90%)
Average block utilization: 60%
```

원인:

1. Block size가 너무 큼
2. 짧은 시퀀스가 많음

해결책:

```
# Block size 감소
vllm serve model_name \
    --block-size 8  # 기본 16 → 8

# 주의: Block size 감소는 관리 오버헤드 증가
# 시퀀스 길이 분포에 따라 조정 필요
```

9.4 Block Table Errors

증상:

```
AssertionError: Block table mismatch
Expected blocks: 10, Got: 8
```

원인:

1. Block table과 실제 블록 수 불일치

2. 메모리 corruption
3. Concurrent access 문제

해결책:

```
# 1. Block size 확인
# block_size가 모든 layer에서 일치하는지 확인

# 2. Prefix caching 비활성화 (디버깅용)
vllm serve model_name \
    --disable-prefix-caching

# 3. CUDA graph 비활성화 (디버깅용)
vllm serve model_name \
    --enforce-eager

# 4. 로그 레벨 증가
VLLM_LOGGING_LEVEL=DEBUG vllm serve model_name
```

---

10. 요약

10.1 PagedAttention 핵심 원리

```
1. Block-based Memory Management
   ┌──────────────────────────────────┐
   | Block 0  Block 1  Block 2  ... |
   | [16 tok] [16 tok] [16 tok]     |
   └──────────────────────────────────┘

2. Virtual-to-Physical Mapping
   Sequence → Block Table → Physical Blocks

3. Dynamic Allocation
   필요한 만큼만 블록 할당

4. Prefix Caching
   동일한 prefix 공유 → Block 재사용
```

10.2 주요 이점

이점	개선 정도
메모리 효율	7-10x 증가
Throughput	2-4x 증가
Batch Size	4-8x 증가
Prefix Cache Hit	~90% 시간 절약

10.3 구현 체크리스트

기본 설정:

- [ ] Block size = 16 (대부분의 경우)
- [ ] Enable prefix caching
- [ ] GPU memory utilization = 0.90
- [ ] Monitor block pool usage

최적화:

- [ ] Block size tuning (시퀀스 길이 분포에 따라)
- [ ] Prefix caching strategy (공통 prompt 최대화)
- [ ] Memory profiling (block utilization 확인)
- [ ] Throughput vs. Latency 균형

모니터링:

- [ ] Block pool free blocks
- [ ] Prefix cache hit rate
- [ ] Memory efficiency
- [ ] Internal fragmentation

참고 자료

- **Paper**: [Efficient Memory Management for Large Language Model Serving with PagedAttention]

- **코드**: `vllm/v1/core/block_pool.py`, `vllm/v1/core/kv_cache_manager.py`, `vllm/attention/ops/paged_attn.py`
- **Blog**: [vLLM: PagedAttention Deep Dive]